Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez
Javier Alejandro Penagos Hernandez
Nicolas Castro Rivera

1 Summary

This work formulates the customer churn retention problem as a Markov Decision Process (MDP). The objective is to train a Reinforcement Learning (RL) agent to select the best retention strategy for a customer with a probability of churn. Although the available dataset does not contain labeled data describing the outcome of different retention strategies, it provides sufficient customer and service information to establish an environment where the agent can learn which actions can reduce the probability of churn.

2 MDP Formulation

The MDP is composed of a state space containing customer features, an action set representing different retention strategies, stochastic transitions based on the customer's acceptance of an offer, and a reward function based on the reduction of the predicted churn probability and the cost of the selected action. RL agent will learn using DQN or PPO to determine the action with the highest expected reward.

2.1 Problem Statement

The objective of the MDP is to determine the best retention strategy for a customer according to their characteristics and current probability of churn. The RL agent receives the customer's state and selects an action from a set of possible retention strategies. The selected action can be accepted or rejected by the customer, introducing stochasticity into the environment.

The customer's churn probability is obtained using a supervised learning model. This probability is used by the RL environment to evaluate whether the selected retention strategy improves the customer's situation. The objective of the agent is therefore to reduce the probability of churn while considering the cost associated with some retention strategies.

The simulation that will represent this MDP, is made based on the KKBox case, for that reason, the states and actions were built with these data. Also the supervised model selected for estimate the probability of churn, has some problems observed in the workshop 1, the actions proposal tried to be generic, but can has bias originated of the importance that supervised model gives to some features, is considered a limitation of this MDP.

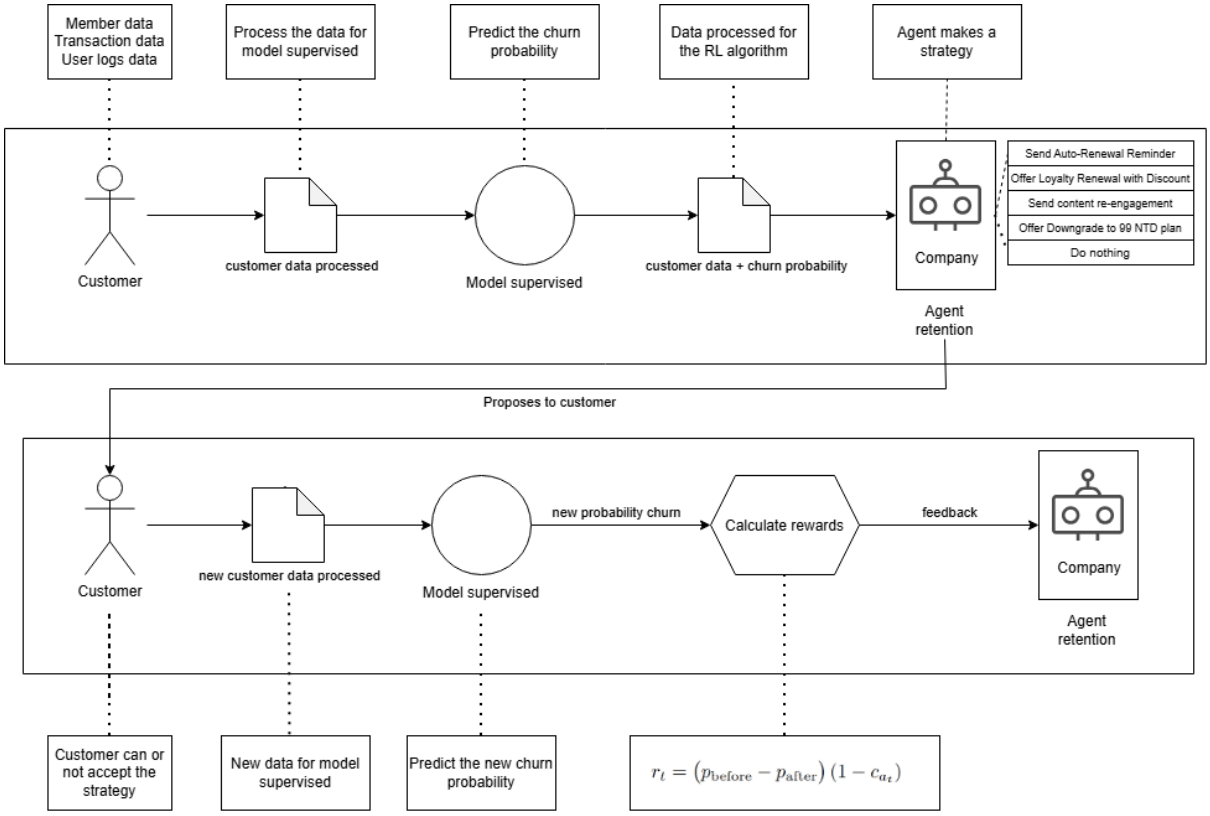


Figure 1: Overview of the retention agent system.

2.2 State Space

The state consist in the customer features, the data processed generated in the preprocessing pipeline. Any row can be a state valid for an episode in the environment and added to the probability of churn. There are 3 sections: member data of the user like age, city, etc. Transaction data calculated with the historical transactions. User logs data calculated with logs in the last 3 months and probability churn.

For this reason, each state is represented as a vector of 60 elements:

```
State = [Members 8 + transactions 29 + user logs 22 + churn probability 1 = 60]
```

2.3 Action Set

The action set are five discrete retention strategies. Each action is designed to target a different groups of customer profiles, the agent is encouraged to learn context-dependent selection. Actions are validated against the customer's current subscription status before being offered, and each is subject to a stochastic acceptance by the customer.

For this simulation is based in a optimiste scenario, because normally the customer not accept the strategies with the high probability these actions are proposal.

2.3.1 Send Auto-Renewal Reminder

Strategy proposal. The customer has disabled auto-renewal and their membership is about to expire (within 15 days). The company sends a targeted communication — push notification or email — reminding the customer to reactivate auto-renewal before the deadline. The strategy is only applicable when `is_auto_renew = 0` and the expiry date is within 15 days of the feature cutoff.

Simulation effect. When accepted, the environment directly sets `is_auto_renew = 1` on the customer's most recent transaction record. No new transaction is appended. The features most directly affected are `last_is_auto_renew` and `autorenew_not_cancel`.

Acceptance probability. The favorable case applies to recently active customers (`days_since_last_log` ≤ 30), who are still engaged and therefore more receptive: **60%** acceptance. For inactive customers the probability drops to **10%**, as they have likely already decided not to renew.

Cost. This action carries a cost of **1%**, reflecting the minimal operational expense of sending a digital communication.

2.3.2 Offer Loyalty Renewal with Discount

Strategy proposal. Long-standing subscribers (account tenure > 730 days, i.e. 2 years) with auto-renewal still active are offered a loyalty reward: a 30-day renewal at the standard list price of 149 NTD with a fixed 20 NTD discount, so the customer pays 129 NTD. The offer acknowledges the customer's loyalty while creating a positive touchpoint before a potential churn event.

Simulation effect. When accepted, a new transaction is appended with `payment_plan_days = 30`, `plan_list_price = 149`, `actual_amount_paid = 129`, and `is_auto_renew = 1`. The features most directly affected are `last_discount`, `average_amount_paid`, and `difference`.

Acceptance probability. The favorable case applies to customers with 15 or more active listening days in the most recent month, indicating high engagement: **90%** acceptance. Customers with lower activity are less motivated by a loyalty offer: **15%** acceptance.

Cost. This action carries a cost of **15%**, representing the 20 NTD discount the company absorbs on each loyalty renewal.

2.3.3 Send Content Re-Engagement

Strategy proposal. Customers whose listening activity has declined month-over-month receive a personalised content recommendation campaign — curated playlists, new releases, or promoted content — aimed at reactivating usage before the subscription lapses. The strategy is only applicable when the song play count in January 2017 is lower than in December 2016.

Simulation effect. When accepted, between 2 and 8 synthetic listening-session log entries are injected into the customer's raw user logs, drawn randomly from a pool of 8 template sessions spread across January 2017. Each entry's song counts are scaled to the customer's December 2016 daily average to preserve individual listening patterns. The features most directly affected are `trend_songs`, `trend_secs`, `last_login`, and `days_since_last_log`.

Acceptance probability. The favorable case applies to recently active customers (`days_since_last_log` ≤ 30), who are still reachable: **80%** acceptance. For customers who have been inactive for longer, engagement is unlikely: **10%** acceptance.

Cost. This action carries a cost of **5%**, representing the expense of executing a personalised content campaign.

2.3.4 Offer Downgrade to 99 NTD Plan

Strategy proposal. Price-sensitive customers are offered a lower-cost 30-day subscription plan at 99 NTD, reducing the financial barrier to renewal. The strategy targets customers currently on a more expensive plan (`plan_list_price` $>$ 99 NTD), prioritising retention over revenue per subscriber.

Simulation effect. When accepted, a new transaction is appended with `payment_plan_days = 30`, `plan_list_price = 99`, `actual_amount_paid = 99`, and `last_discount = 0`. The features most directly affected are `last_plan_days`, `average_amount_paid`, `last_discount`, `sum_plan_days`, and `average_plan`.

Acceptance probability. The favorable case applies to newer customers (account tenure $<$ 365 days), who are less committed to the current plan and more willing to switch tiers: **90%** acceptance. For longer-tenure customers, the downgrade is less appealing as they are already accustomed to the existing plan: **15%** acceptance.

Cost. This action carries a cost of **15%**, reflecting the permanent reduction in revenue per subscriber that results from moving to a cheaper plan.

2.3.5 Do Nothing

Strategy proposal. No retention intervention is applied. The agent may select this action when no other strategy is expected to yield a positive outcome, or when budget conservation takes priority.

Simulation effect. The customer's state and raw data remain completely unchanged.

Acceptance probability. This action is always executed with probability **100%**; the stochastic acceptance step is skipped entirely.

Cost. This action has zero cost.

2.4 Transition Dynamics

The selected action directly affects the customer's state. However, before modifying the state, the environment determines whether the customer accepts or rejects the offered strategy. This makes the transition stochastic because the same action can produce different outcomes.

The acceptance probabilities are defined according to the type of retention strategy:

- Send Auto-Renewal Reminder: 60% (favorable) / 10% (unfavorable).
- Offer Loyalty Renewal with Discount: 90% (favorable) / 15% (unfavorable).
- Send Content Re-Engagement: 80% (favorable) / 10% (unfavorable).
- Offer Downgrade to 99 NTD Plan: 90% (favorable) / 15% (unfavorable).

- Do Nothing: 100% (always accepted, no state change).

The transition function can be represented as:

$$s_{t+1} = f(s_t, a_t) \tag{1}$$

The acceptance or rejection of an action is stochastic. However, once an action is accepted, the modification applied to the customer's state is deterministic. Therefore, the same state and the same accepted action produce the same resulting state.

2.5 Reward Function

The reward is the reduction in predicted churn probability discounted by the action cost, the agent search maximize the reward, if the reduction difference of probabilities is high the reward will be increase. But some strategies can generate cost to company and can penalize the use of this for retain de the customer.

$$r_t = \left(p_{\text{before}} - p_{\text{after}}\right)\left(1 - c_{a_t}\right) \tag{2}$$

where p_{before} and p_{after} are the supervised model's churn probability estimates before and after the action is applied, and $c_{a_t} \in \{0,\, 0.10,\, 0.15\}$ is the cost of the selected action. When an action is rejected the state is unchanged and $r_t = 0$.

2.6 Episode

An episode starts with a customer from the KKBox dataset. The customer's initial state is provided to the supervised learning model, which predicts the probability of churn. This information is then provided to the RL agent.

The agent select the retention strategy with the highest expected reward. The selected action is then evaluated by the environment, which determines whether the customer accepts or rejects the offer according to the corresponding acceptance probability.

If the customer accepts the offer, the action modifies the customer's state. The supervised model then predicts the new probability of churn, which is compared with the probability before the action to calculate the reward. The episode terminates after this reward is obtained.

If the customer rejects the offer, the customer's state remains unchanged and the reward is zero. The episode then terminates.

3 Environment

The environment is implemented as a `gymnasium.Env` subclass in `workshops/w3/environment/env.py` (`KKBoxRetentionEnv`).

3.1 Environment Setup

The action space is `Discrete(5)`, one index per retention strategy. The observation space is a `Box` of shape $(60,)$ in `float32`, composed of the 59 preprocessed customer features followed by the current churn probability estimate.

Four auxiliary classes support the environment:

- `CustomerSelector` (`supervised_model/customer_selector.py`): builds a pool of up to 1,500 churned users sampled from the February cohort. It reads member data, transactions, and user logs from the raw Parquet files and serialises the pool to disk so subsequent runs skip the expensive I/O.
- `PreprocessorCustomerData` (`supervised_model/preprocessor_customer_data.py`): replicates the Workshop 1 pipeline on a single customer dict, computing all 59 features from raw member, transaction, and log records. It loads the clipping parameters and imputation values from the `development.json` artefact produced during training, ensuring identical feature engineering at inference time.
- `SupervisedModel` (`supervised_model/supervised_model.py`): wraps the M3 XGBoost model saved in `workshops/w1/artifacts/sherlock_w1/`. It exposes `predict_probability()`, which returns the model's churn probability for a preprocessed feature row.
- `ActionsManager` (`actions_manager.py`): validates which actions are applicable to the current customer state, evaluates the stochastic acceptance check, and applies the accepted action to the raw customer dict. Action metadata (name, acceptance probabilities, cost) is declared in `constants.py` as a tuple of `RetentionAction` objects.

On construction the environment pre-computes an observation cache: every customer in the pool is run through the preprocessor and the supervised model once and the resulting $(\mathit{observation}, p_{\text{churn}})$ pairs are stored. This avoids re-featurizing on every `reset()` call.

3.2 Episode Initialization — `reset()`

Each call to `reset()` selects one customer uniformly at random from the pool using the environment's seeded NumPy RNG. The cached observation and initial churn probability for that customer are retrieved directly without re-running the preprocessor. The customer's raw dict is stored in `self.customer_data` and `self.probability_churn_before` is set for use in the subsequent `step()` call. The method returns the 60-dimensional observation vector.

3.3 Transition, Reward, and Termination — `step(action)`

`step(action)` executes the selected retention strategy and terminates the episode immediately (single-step episodes, `terminated = True`).

1. **Action execution.** `ActionsManager.apply_action()` is called with the current customer dict and the selected action index. Internally it checks validity, draws a Bernoulli sample against the acceptance probability, and — if accepted — modifies the raw customer dict deterministically (appending a transaction and, for the re-engagement action, log entries). It returns the updated dict and the action cost (0.0 if rejected).

2. **Re-featurization.** The updated dict is passed to `PreprocessorCustomerData.preprocess_customer()`, which recomputes all 59 features from scratch. The supervised model then estimates p_{after} from the new feature row.

3. **Reward.** Computed as $r = (p_{\text{before}} - p_{\text{after}})\,(1 - c_a)$, where c_a is the action cost. A positive reward results from a reduction in churn probability; cost penalises expensive strategies.

4. **Termination.** The episode ends unconditionally after the single step. The `info` dict returns the before/after probabilities, the action index, and the cost for logging and evaluation.

3.4 Random Policy Baseline

Before training any agent, a random policy verifies that the environment behaves correctly under unconstrained exploration. The policy selects uniformly at random among the valid actions for each customer and runs for 100 single-step episodes. The baseline is implemented in `workshops/w3/baselines/random_policy.py`:

```
python workshops/w3/baselines/random_policy.py --episodes 100 --seed 42
```

Two runs under different seeds confirm correct behaviour (Table 1). Both seeds confirm that all five retention strategies were selected at least once, validating that the action masking reaches every branch. Acceptance rates around 24–29% are consistent with the low base acceptance probabilities. The mean churn reduction of 2–4 percentage points is expected from an unguided policy on a high-churn pool.

Metric	**Seed 42**	**Seed 123**
Accepted actions	24 / 100	29 / 100
Mean episode return	0.0411	0.0238
Mean initial churn prob	0.8461	0.8253
Mean final churn prob	0.8021	0.8004
Mean probability change	0.0440	0.0249
Total action cost	2.41	3.30
All actions occurred	Yes	Yes
Episode length	1 step	1 step

Table 1: Random policy diagnostics over 100 episodes.

4 Agent Algorithm

Both DQN and PPO were implemented and compared. Both algorithms handle the discrete action space of 5 retention strategies over a 60-dimensional observation. DQN was selected as the canonical method for discrete spaces, while PPO was included to evaluate whether an actor-critic approach with explicit action masking could exploit the probabilistic structure of acceptance more effectively.

4.1 Deep Q-Network (DQN)

DQN is a value-based algorithm that approximates the action-value function $Q(s, a)$ using a neural network. For each customer state the network outputs one Q-value per action, representing the expected cumulative reward. The agent selects the action with the highest Q-value, directly learning which retention strategy yields the greatest expected return from the current customer state.

Implemented in `workshops/w3/agent/agent_dqn.py`.

Table 2: DQN active hyperparameters (`DQNConfig`).

Parameter	Description	Value
Hidden size	Units in each hidden layer of the Q-network	128
Batch size	Transitions sampled from the replay buffer per training step	64
Replay buffer capacity	Maximum transitions stored for experience replay	50,000
Gradient clip	Maximum gradient norm to prevent training instability	10.0
ε start / min / decay	Exploration schedule: initial value, floor, and per-step multiplier	1.0 / 0.05 / 0.995

The discount factor γ and the target network update interval are excluded because every episode terminates after one step (`terminated = True`), so the Bellman target reduces to $y = r$ and the target network's Q-values are multiplied by zero and never used.

- `QNetwork` (`agent_dqn.py`): fully-connected network with two hidden layers of 128 units and ReLU activations, mapping the 60-dimensional state to 5 Q-values, one per action.
- `ReplayBuffer` (`agent_dqn.py`): fixed-capacity FIFO buffer (50,000 transitions) storing (state, action, reward, next_state, terminated, next_valid_actions) tuples, sampled uniformly at random each training step.
- `DQNAgent.select_action` (`agent_dqn.py`): applies an action mask over the valid actions for the current customer. During training, ε-greedy exploration is restricted to those valid actions.
- `DQNAgent.train_step` (`agent_dqn.py`): implements Double DQN — the online network selects the best next action and the target network evaluates its Q-value. Loss is Smooth L1.

4.2 Proximal Policy Optimization (PPO)

PPO is a policy gradient algorithm that directly optimizes the policy $\pi(a \mid s)$ — the probability distribution over actions given the current state. Rather than estimating Q-values explicitly, PPO updates the policy to increase the probability of actions that yielded above-average returns, while clipping large updates to keep training stable. An actor network outputs action probabilities and a critic network estimates the state value to compute the advantage.

Implemented in `workshops/w3/agent/agent_ppo.py`.

Table 3: PPO active hyperparameters (`PPOConfig`).

Parameter	Description	Value
Hidden size	Units in each hidden layer of the actor-critic network	128
Clip ε	Clipping threshold for the PPO surrogate objective ratio	0.2
Entropy coefficient	Weight of the entropy bonus to encourage exploration	0.01
Value coefficient	Weight of the value loss relative to the policy loss	0.5
Update epochs	Gradient passes over each collected rollout batch	4
Minibatch size	Transitions per gradient update within each epoch	64
Gradient clip	Maximum gradient norm to prevent training instability	0.5
Rollout size	On-policy episodes collected before each PPO update	128

The discount factor γ and GAE λ are excluded because in single-step episodes the return equals r directly and the advantage reduces to $A = r - V(s)$ regardless of λ, so neither parameter has any effect on training.

- `ActorCriticNetwork` (`agent_ppo.py`): shared MLP trunk (60→128→128, Tanh) branching into a policy head (128→5 logits) and a value head (128→1 scalar).
- `PPOAgent.select_action` (`agent_ppo.py`): invalid actions are masked by setting their logits to $-\infty$ before constructing the Categorical distribution. In evaluation mode the action with the highest masked logit is selected deterministically.
- `PPOAgent.update` (`agent_ppo.py`): receives a completed rollout batch, computes one-step advantages ($A = r - V(s)$), and runs 4 epochs of minibatch updates using the PPO clipped objective plus an entropy bonus (coefficient 0.01).

4.3 Hyperparameter Search

Three named configurations were evaluated for each algorithm, varying learning rate, exploration or regularisation strength to cover fast, balanced, and conservative update regimes. All other hyperparameters were held fixed. Learning rate was chosen as the primary axis because it is the most sensitive hyperparameter in both value-based and policy gradient methods. For DQN, ε decay and weight decay were co-varied to keep exploration duration and regularisation consistent with each update speed.

4.3.1 DQN Configurations

Table 4: DQN hyperparameter configurations and search results.

Configuration	Learning Rate	ε decay	Weight Decay	Best Val. Return	Best Episode
`exploratory`	1×10^{-3}	0.999	0	+0.1060	1250
`balanced`	5×10^{-4}	0.995	10^{-5}	+0.0771	1000
`conservative`	1×10^{-4}	0.990	10^{-4}	+0.0628	1250

4.3.2 PPO Configurations

Table 5: PPO hyperparameter configurations and search results.

Configuration	Learning Rate	Weight Decay	Best Val. Return	Best Episode
`exploratory`	1×10^{-3}	0	+0.1041	750
`balanced`	3×10^{-4}	10^{-5}	+0.1126	2000
`conservative`	1×10^{-4}	10^{-4}	+0.1056	1750

4.4 Training Protocol

Training is implemented in `workshops/w3/train/train_dqn.py` and `workshops/w3/train/train_ppo.py`. Both scripts share the same infrastructure: a pool of 1,500 churned customers is split 80/20 into training (1,200) and validation (300) sets. Each configuration trains for 3,000 episodes. Validation runs every 250 episodes over 100 held-out customers without exploration, and the checkpoint with the highest mean validation return is saved.

Environment–agent interaction. Each episode begins with `env.reset()`, which samples one customer from the training pool and returns the 60-dimensional observation. The agent receives this state, queries `ActionsManager` for the list of valid actions, and selects one. The selected action is passed to `env.step()`, which executes the acceptance check, updates the customer's raw data if accepted, re-featurizes via the preprocessor, re-scores via the supervised model, and returns the next observation and reward. The episode terminates immediately after this single step. The agent uses the (state, action, reward, next_state) tuple to update its parameters.

DQN training loop. At each step the agent selects an action using ε-greedy exploration over the valid actions, storing the transition in the replay buffer. Once the buffer holds enough transitions, a random minibatch of 64 is sampled and the online network is updated by minimising the Smooth L1 loss against the Double DQN target. Epsilon decays from 1.0 to 0.05 across episodes, shifting the agent from exploration to exploitation. Figure 2 shows the return curve for the best configuration (`lr_1e-3`): after an exploratory phase with high variance up to episode 1,250, the mean return stabilises around 0.10–0.11 and validation scores confirm the improvement.

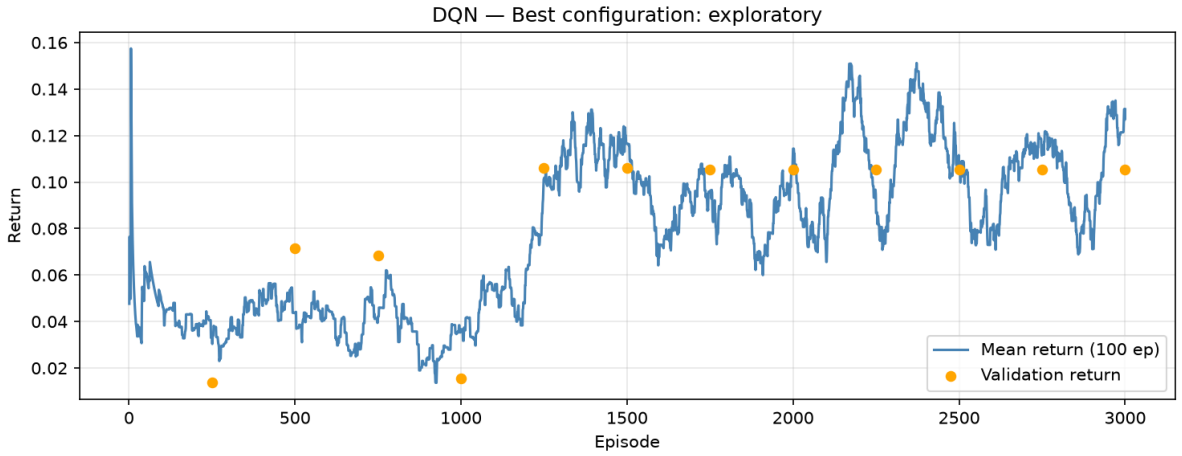


Figure 2: DQN best configuration (`lr_1e-3`): rolling mean return (100 episodes) and validation return over 3,000 training episodes.

PPO training loop. The agent collects transitions across 128 episodes into an on-policy rollout buffer. For each transition the actor samples an action from the masked Categorical distribution and the critic estimates the state value. Once the rollout is complete, one-step advantages ($A = r - V(s)$) are computed and the agent runs 4 epochs of minibatch updates using the PPO clipped surrogate objective plus a value loss and entropy bonus. Figure 3 shows the return curve for the best configuration (`lr_3e-4`): similarly to DQN, returns stabilise around 0.10–0.13 after initial variance. Figure 4 shows the corresponding losses: policy loss remains near zero (clipping is active), value loss oscillates at low magnitude, and entropy starts high and gradually decreases as the policy concentrates on preferred actions.

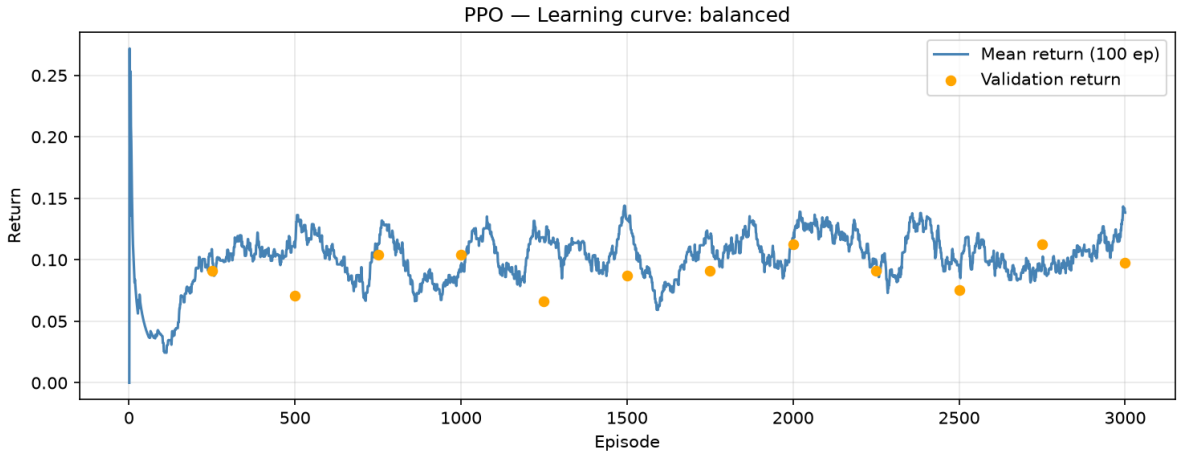


Figure 3: PPO best configuration (`lr_3e-4`): rolling mean return (100 episodes) and validation return over 3,000 training episodes.

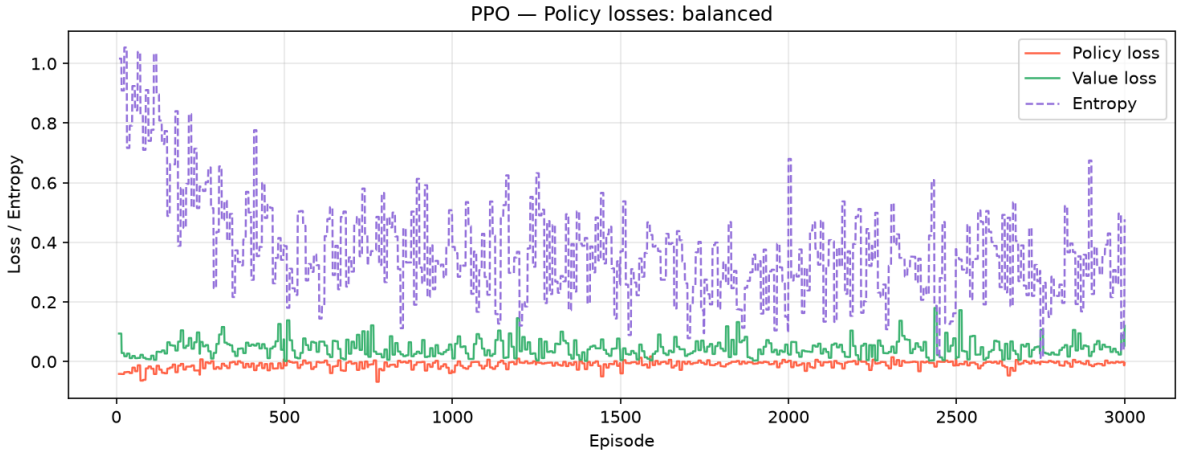


Figure 4: PPO best configuration (`lr_3e-4`): policy loss, value loss, and entropy over 3,000 training episodes.

5 Sample trajectory visualizations

6 Discussion

7 Conclusion

References

[1] Ahmad, A. K., Jafar, A., & Aljoumaa, K. (2019). Customer churn prediction in telecom using machine learning in big data platform. *Journal of Big Data*, 6, 28.

[2] Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. *European Journal of Operational Research*, 218(1), 211–229.

[3] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[4] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

[5] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2623–2631).

[6] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

[7] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

[8] Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning* (pp. 233–240).